

实验报告

课程: 机器学习 实验名称: Python 编程练习 日期:3.9

姓名:雷文豪 学号:138022024031 专业班级:人工智能



1 实验目的

1. Python 基本语法与编程练习，相关模块的使用练习。
2. 对待求解问题，通过设计代码实施解题，并完成程序正确性的验证。

2 实验内容提要与实验设备

1. 提供示例代码，涉及 Python 基本语法、相关模块、相关编程的内容。
2. 对待求解问题，设计求解方法、编写程序实施求解。
3. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

报告填写提示：提供实验结果的截图，程序输出数字文字时只截图有数字文字的部分，提供清晰的小截图；若插入本文档的截图仍比较大，将截图拉小点，再进行一次截图。

提示格式：你的代码须使用黑色，且不使用任何底色、任何修饰、截图，否则老师看着很吃力；若你想让老师看到少量重点文字，把文字标成蓝色。

4 练习示例代码

4.1 列表操作

```
1  ## 一维列表操作
2  alist = [0.3, 'hello', True] # 直接创建列表
3  b = [i for i in range(1,11) if i%2==0] # 列表生成式
4  res = alist[0:2] # 列表的切片, 左闭右开
5  res = alist[-1] # 负索引-从尾向首的顺序
6  alist.append("hello world") # 新增元素
7  del alist[:2] # 删除元素
8  for k in alist: # 循环
9      print(k)
10
11 ## 二维列表操作
12 B1 = [[6,5,4],[7,8,9]] # 直接创建
13 B2 = [['张',21,86], ['李',19,73]]
14 print(B2[0][1])
```

4.2 字符串的文件读写

```
1  ## 字符串写入文件, 从文件读出字符串
2  string1 = 'A big house\n' # 末尾换行符
3  string2 = 'It is in the village'
4  f = open('myfile.txt','a+') # 追加 a+, 写方式 w
5  f.write(string1) # 将字符串追加写入文件
6  f.write(string2)
7  f.close() # 关闭文件
8
9  f=open('myfile.txt', 'r') # 以读的方式打开文件
10 line = f.readline() # 读入一行数据作为字符串
11 a_list = []
12 while line:
13     scut = line.split() # 按分隔符切字符串, 转列表
14     a_list.append(scut) # 列表每个元素对应一行数据
15     line = f.readline()
16 f.close()
```

4.3 使用模块 `numpy` 创建数组及操作

```

1  ## 使用模块 numpy 创建数组及操作
2  import numpy as np
3  a = np.arange(10) # 创建一维数组
4  b = a.copy() # 创建副本, 非共享
5  a[2:5] # 索引切片
6
7  ## 使用模块 numpy 创建 3 行 4 列的多维数组
8  A = np.random.random((3,4)) # 随机
9  B = np.arange(12).reshape(3,4) # 元素 0-11
10 B1 = np.ones((4,3), dtype=float) # 元素全 1
11 B1[0:2,0] = 8.3 # 索引切片
12 B1.shape # 维数信息
13 B2 = A[1,:] # 取行数据, 与 A 共享存储空间
14 B3 = A[1,:].copy() # 取行数据, 非共享
15
16 ## 使用模块 numpy 多维数组的运算
17 C = A * B # 同维数组的元素点乘
18 B2 = np.transpose(B) # 数组的转置
19 B2 = B.T # 数组的转置
20 D = A.dot(B2) # 2 个数组的矩阵乘法
21 E = np.dot(A, B2)
22 F = np.add(B, C) # 矩阵相加
23 G = B + C # 矩阵相加
24 B.sum(axis=0) # 按列求和
25 B.sum(axis=1) # 按行求和
26 B.max()
27 B1 = np.max(B,0) # 求矩阵每列元素最大值

```

5 编写代码求解问题

5.1 寻找成绩满足要求的学生

从文件（成绩表简课.txt）读入学生信息与成绩，然后寻找其中成绩满足要求的学生（即某学生 2 门课的平均成绩 90 分及以上），将找到的学生信息存入列表，再打印输出列表中的学生信息及其成绩，最后存入文件（找到信息.txt）。

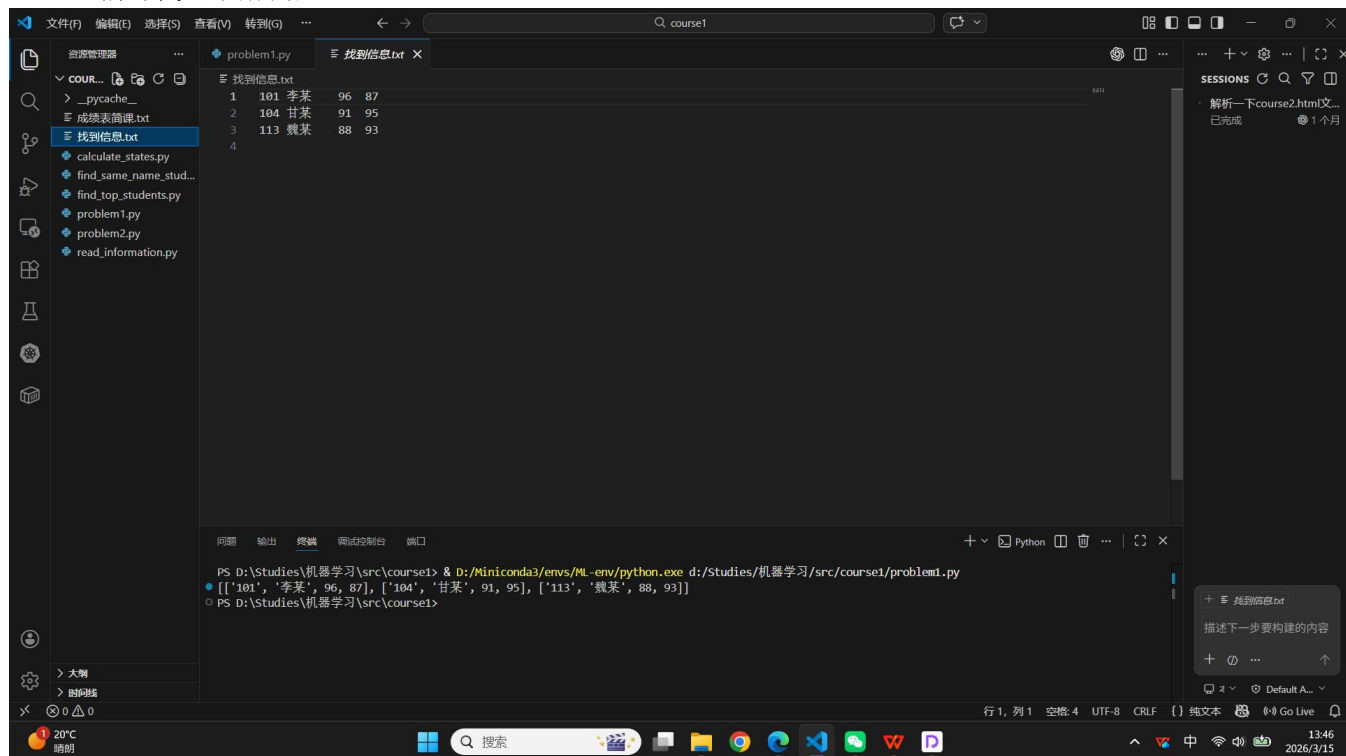
要求：（1）编程实现上述工作，写出全部代码（非图片，添加必要的注释说明）。自己设计方法实现，不调用库函数；（2）程序成功的截图（包含源文件名 <字母 + 学号后 3 位>、运行光标在程序尾

行)。

```
1 # 核心代码：读取文件并筛选平均分 90 分以上的学生
2 def find_top_students():
3     top_students = []
4     [Coding here ...]
5     # 寻找两门课平均成绩超过90分的学生
6     for data in student_info_set:
7         score_average = (data[2] + data[3])/2
8         if(score_average >= 90):
9             student_info = [info for info in data]
10            top_students.append(student_info)
11
12    return top_students
```

运行结果截图:

```
PS D:\Studies\机器学习\src\course1> & D:/Miniconda3/envs/ML-env/python.exe d:/Studies/机器学习/src/course1/problem1.py
[ ['101', '李英', 96, 87], ['104', '甘菜', 91, 95], ['113', '袁东', 88, 93] ]
PS D:\Studies\机器学习\src\course1>
```



5.2 课程情况统计

从文件（成绩表简课.txt）读入学生信息与成绩，然后创建 numpy 多维数组存入读入的姓名与成绩，求每门课的最高分与最低分，求每门课的及格率，计算每门课的平均分与标准差（见公式（1）），打印输出求解结果。再找出重名学生信息，并输出重名学生信息与成绩。

$$\sigma = \sqrt{\frac{1}{N} \sum_{n=1}^N (x^{(n)} - \mu)^2}, \quad (1)$$

其中 N 是样本数量； $x^{(n)}$ 是第 n 个样本点的数值； μ 是样本均值。

要求：（1）编程实现上述工作，写出全部代码（非图片，添加必要的注释说明）。自己设计方法实现，不调用库函数；（2）程序成功的截图（包含源文件名 <字母 + 学号后 3 位>、运行光标在程序尾行）。

```

1 # 核心代码 1：计算各科统计指标函数
2 def calculate_stats(subject_scores):
3     [Coding here ...]
4     # 只提取成绩部分，并转换为数值类型
5     score_matrix = np.array([student[2:] for student in student_info_set], dtype=int)
6
7     # 求最高分与最低分（只对成绩列计算）
8     max_scores = np.max(score_matrix, axis=0)
9     min_scores = np.min(score_matrix, axis=0)
10
11     # 求各科的及格率
12     pass_rate = np.mean(score_matrix >= 60, axis=0)

```

求各科的平均分与标准差

```
mean_scores = np.mean(score_matrix, axis=0)
```

```
std_scores = np.std(score_matrix, axis=0)
```

```
4     return min_val, max_val, mean, std_dev, pass_rate
```

5

6 # 核心代码 2: 如何找出重名的学生?

编写一个函数, 用于寻找重名的学生信息

```
def find_same_name_student_info(student_info_set):
```

```
    # 将学生姓名单独提取
```

```
    names = [name[1] for name in student_info_set]
```

```
    # 寻找重名的学生, 使用集合
```

```
    seen = set()
```

```
    duplicate_names = set()
```

```
    for name in names:
```

```
        if name in seen:
```

```
            duplicate_names.add(name)
```

```
        else:
```

```
            seen.add(name)
```

```
    # 返回重名学生的信息
```

```
    return [student for student in student_info_set if student[1] in
duplicate_names]
```

注: 使用功能模块化设计, 最后集成到一个.py文件中运行, 在设计过程中为方便某些操作返回值有所改变,

经验证后不影响结果的正确性, 不足之处是代码的健壮性有待增强

运行结果截图:

```

def main():
    # 语文成绩统计指标
    print("\n语文成绩统计指标: \n")
    print(f"最高分: {states['max_scores'][0]:.2f}")
    print(f"最低分: {states['min_scores'][0]:.2f}")
    print(f"及格率: {states['pass_rate'][0]:.2%}")
    print(f"平均分: {states['mean_scores'][0]:.2f}")
    print(f"标准差: {states['std_scores'][0]:.2f}")

    # 数学成绩统计指标 (索引1)
    print("\n数学成绩统计指标: \n")
    print(f"最高分: {states['max_scores'][1]:.2f}")
    print(f"最低分: {states['min_scores'][1]:.2f}")
    print(f"及格率: {states['pass_rate'][1]:.2%}")
    print(f"平均分: {states['mean_scores'][1]:.2f}")
    print(f"标准差: {states['std_scores'][1]:.2f}")

    # 寻找重名的学生
    same_name_students = find_same_name_student_info.find_same_name_student_info(student_info_set)
    print("重名学生信息: ")
    print(same_name_students)

if __name__ == "__main__":
    main()

```

PS D:\Studies\机器学习\src\course1> & D:\Miniconda3\envs\ML-env\python.exe d:\Studies\机器学习\src\course1\problem2.py
 最高分: 95.00
 最低分: 47.00
 及格率: 88.89%
 平均分: 77.33
 标准差: 13.50
 重名学生信息:
 [['107', '胡某', 73, 92], ['108', '刘某', 54, 65], ['115', '刘某', 84, 63], ['117', '胡某', 78, 85]]

5.3 选做，任选 1 个小题，开放题

1. 试量化分析 2 门课成绩之间的相关性。
2. 试分析每门课成绩的分布情况，绘图展示，并讨论是否符合正态分布。

要求：（1）编程实现上述工作，写出全部代码（非图片，添加必要的注释说明）。（2）程序成功的截图（包含源文件名 <字母+学号后 3 位>、运行光标在程序尾行）。

源码：

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
from scipy.stats import norm
import read_information
```

编写一个函数实现量化分析两门课成绩之间的相关性，参数是student_info_set包含学生信息，返回值correlation_result包含相关性分析结果的字典

```
def analyze_correlation(student_info_set):
    # 提取语文和数学成绩
    yuwen_scores = np.array([student[2] for student in student_info_set])
    math_scores = np.array([student[3] for student in student_info_set])

    # 计算皮尔逊相关系数
    pearson_corr = np.corrcoef(yuwen_scores, math_scores)[0, 1]

    # 计算斯皮尔曼相关系数
    spearman_corr = stats.spearmanr(yuwen_scores, math_scores)[0]

    # 计算协方差
    covariance = np.cov(yuwen_scores, math_scores)[0, 1]

    # 创建散点图
    plt.figure(figsize=(10, 6))
    plt.scatter(yuwen_scores, math_scores, alpha=0.7, color='blue', s=80)

    # 添加回归线
    z = np.polyfit(yuwen_scores, math_scores, 1)
    p = np.polyd(z)
    plt.plot(yuwen_scores, p(yuwen_scores), "r--", alpha=0.8, linewidth=2)
```

```

# 设置图表属性
plt.xlabel('语文成绩', fontsize=12)
plt.ylabel('数学成绩', fontsize=12)
plt.title(f'语文与数学成绩相关性分析\n皮尔逊相关系数: {pearson_corr:.3f}',
fontsize=14)

plt.grid(True, alpha=0.3)

# 添加每个点的标签
for i, student in enumerate(student_info_set):
    plt.annotate(student[1], (yuwen_scores[i], math_scores[i]),
                  xytext=(5, 5), textcoords='offset points', fontsize=8)

plt.tight_layout()
plt.show()

# 返回分析结果
correlation_result = {
    'pearson_correlation': pearson_corr,
    'spearman_correlation': spearman_corr,
    'covariance': covariance,
    'yuwen_scores': yuwen_scores,
    'math_scores': math_scores
}

return correlation_result

# 编写一个函数实现分析每门课成绩的分布情况，绘图展示，并讨论是否符合正态分布，参数
student_info_set是学生信息列表，返回值distribution_result包含分布分析结果的字典
def analyze_distribution(student_info_set):
    # 提取成绩数据
    yuwen_scores = np.array([student[2] for student in student_info_set])
    math_scores = np.array([student[3] for student in student_info_set])

    # 创建子图
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))

    # 分析语文成绩分布

```

```
analyze_single_subject_distribution(yuwen_scores, '语文成绩', axes[0, 0], axes[0,
1])

# 分析数学成绩分布
analyze_single_subject_distribution(math_scores, '数学成绩', axes[1, 0], axes[1,
1])

plt.tight_layout()
plt.show()

# 进行正态性检验
yuwen_normality = perform_normality_test(yuwen_scores, '语文成绩')
math_normality = perform_normality_test(math_scores, '数学成绩')

# 返回分析结果
distribution_result = {
    'yuwen_distribution': {
        'scores': yuwen_scores,
        'mean': np.mean(yuwen_scores),
        'std': np.std(yuwen_scores),
        'normality_test': yuwen_normality
    },
    'math_distribution': {
        'scores': math_scores,
        'mean': np.mean(math_scores),
        'std': np.std(math_scores),
        'normality_test': math_normality
    }
}

return distribution_result

# 编写一个函数实现分析单个科目的分布情况，参数scores是成绩数组，subject_name是科目名称，
hist_ax是直方图子图，qq_ax是Q-Q图子图，返回值normality_result包含检验结果的字典
def analyze_single_subject_distribution(scores, subject_name, hist_ax, qq_ax):
    # 直方图与正态分布曲线
    hist_ax.hist(scores, bins=8, density=True, alpha=0.7, color='skyblue',
edgecolor='black')
```

```

# 添加正态分布曲线
mu, std = np.mean(scores), np.std(scores)
xmin, xmax = hist_ax.get_xlim()
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, mu, std)
hist_ax.plot(x, p, 'k', linewidth=2, label=f'正态分布( $\mu={\mu:.1f}$ ,  $\sigma={\text{std}:.1f}$ )')

hist_ax.set_title(f'{subject_name}分布直方图', fontsize=12)
hist_ax.set_xlabel('分数')
hist_ax.set_ylabel('密度')
hist_ax.legend()
hist_ax.grid(True, alpha=0.3)

# Q-Q图
stats.probplot(scores, dist="norm", plot=qq_ax)
qq_ax.set_title(f'{subject_name} Q-Q图', fontsize=12)
qq_ax.grid(True, alpha=0.3)

# 编写一个函数实现执行正态性检验，参数scores是成绩数组，subject_name是科目名称，返回值
normality_result包含检验结果的字典
def perform_normality_test(scores, subject_name):
    # Shapiro-Wilk检验
    shapiro_stat, shapiro_p = stats.shapiro(scores)

    # Kolmogorov-Smirnov检验
    ks_stat, ks_p = stats.kstest(scores, 'norm', args=(np.mean(scores),
np.std(scores)))

    # 判断是否符合正态分布（显著性水平  $\alpha=0.05$ ）
    is_normal_shapiro = shapiro_p > 0.05
    is_normal_ks = ks_p > 0.05

    normality_result = {
        'shapiro_wilk': {
            'statistic': shapiro_stat,
            'p_value': shapiro_p,
            'is_normal': is_normal_shapiro

```

```

    },
    'kolmogorov_smirnov': {
        'statistic': ks_stat,
        'p_value': ks_p,
        'is_normal': is_normal_ks
    }
}

```

打印检验结果

```

print(f"\n{subject_name} 正态性检验结果:")
print(f"Shapiro-Wilk检验: 统计量={shapiro_stat:.4f}, p值={shapiro_p:.4f}")
print(f"Kolmogorov-Smirnov检验: 统计量={ks_stat:.4f}, p值={ks_p:.4f}")

```

```

if is_normal_shapiro and is_normal_ks:
    print(f"结论: {subject_name} 成绩符合正态分布")
else:
    print(f"结论: {subject_name} 成绩不符合正态分布")

```

```

return normality_result

```

编写一个函数实现主函数执行完整的分析

```

def main():
    # 读取学生数据
    file_path = "成绩表简课.txt"
    student_info_set = read_information.read_student_data(file_path)

    print("=" * 60)
    print("成绩相关性分析")
    print("=" * 60)

    # 执行相关性分析
    correlation_result = analyze_correlation(student_info_set)

    print(f"\n相关性分析结果:")
    print(f"皮尔逊相关系数: {correlation_result['pearson_correlation']:.4f}")
    print(f"斯皮尔曼相关系数: {correlation_result['spearman_correlation']:.4f}")
    print(f"协方差: {correlation_result['covariance']:.4f}")

```

```
# 解释相关系数
corr_value = abs(correlation_result['pearson_correlation'])
if corr_value < 0.3:
    strength = "弱相关"
elif corr_value < 0.7:
    strength = "中等相关"
else:
    strength = "强相关"

if correlation_result['pearson_correlation'] > 0:
    direction = "正"
else:
    direction = "负"

print(f"\n相关性解释: {direction} {strength}")

print("\n" + "=" * 60)
print("成绩分布分析")
print("=" * 60)

# 执行分布分析
distribution_result = analyze_distribution(student_info_set)

# 打印分布统计信息
yuwen_stats = distribution_result['yuwen_distribution']
math_stats = distribution_result['math_distribution']

print(f"\n语文成绩统计:")
print(f"平均值: {yuwen_stats['mean']:.2f}")
print(f"标准差: {yuwen_stats['std']:.2f}")
print(f"偏度: {stats.skew(yuwen_stats['scores']):.4f}")
print(f"峰度: {stats.kurtosis(yuwen_stats['scores']):.4f}")

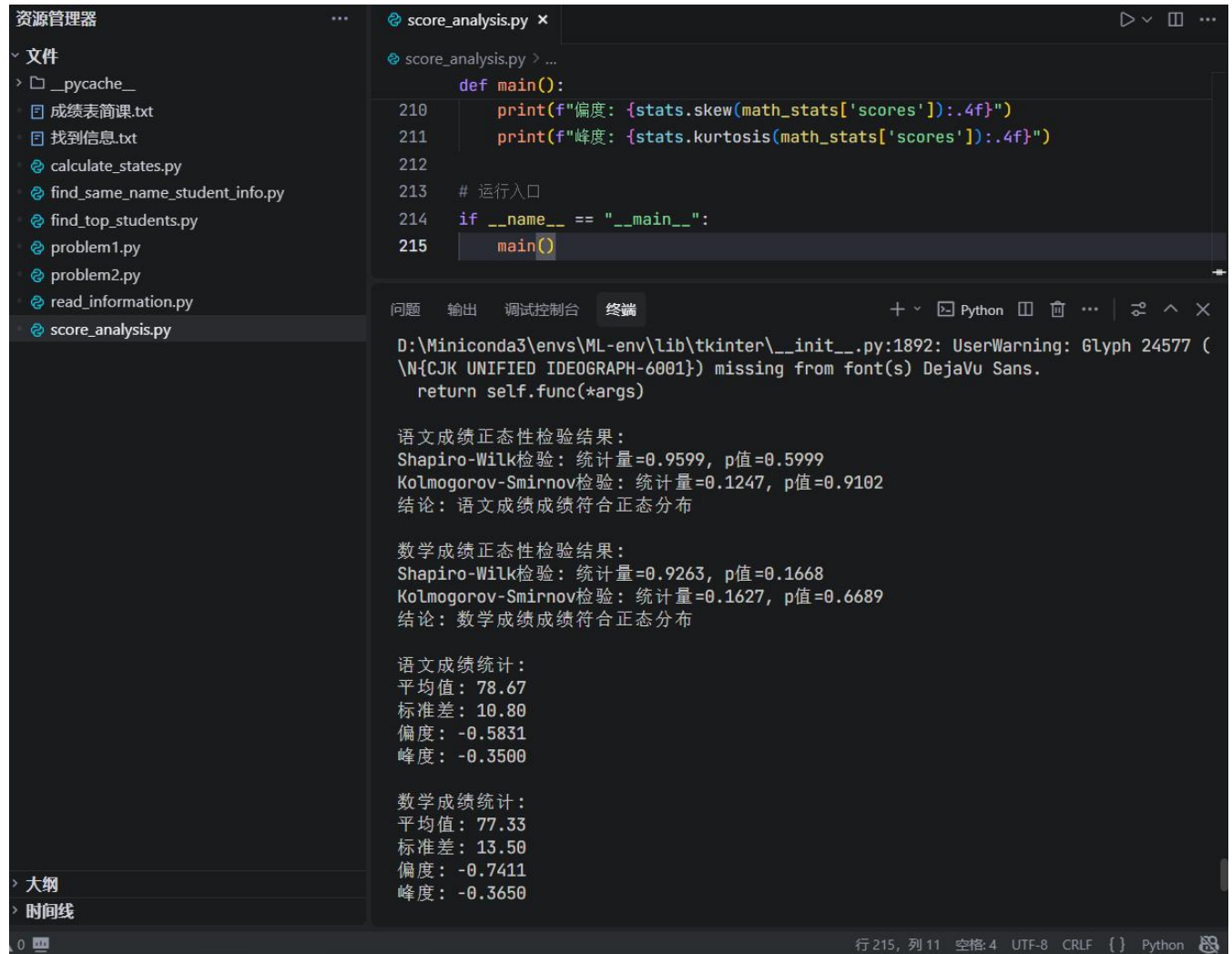
print(f"\n数学成绩统计:")
print(f"平均值: {math_stats['mean']:.2f}")
print(f"标准差: {math_stats['std']:.2f}")
```

```
print(f"偏度: {stats.skew(math_stats['scores']):.4f}")  
  
print(f"峰度: {stats.kurtosis(math_stats['scores']):.4f}")
```

运行入口

```
if __name__ == "__main__":  
    main()
```

运行结果截图:



The screenshot shows a Python IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'score_analysis.py' with several files. The code editor shows the following code:

```
def main():  
    print(f"偏度: {stats.skew(math_stats['scores']):.4f}")  
    print(f"峰度: {stats.kurtosis(math_stats['scores']):.4f}")  
  
# 运行入口  
if __name__ == "__main__":  
    main()
```

The terminal output shows the following results:

```
D:\Miniconda3\envs\ML-env\lib\tkinter\__init__.py:1892: UserWarning: Glyph 24577 (\\N{CJK UNIFIED IDEOGRAPH-6001}) missing from font(s) DejaVu Sans.  
    return self.func(*args)  
  
语文成绩正态性检验结果:  
Shapiro-Wilk检验: 统计量=0.9599, p值=0.5999  
Kolmogorov-Smirnov检验: 统计量=0.1247, p值=0.9102  
结论: 语文成绩成绩符合正态分布  
  
数学成绩正态性检验结果:  
Shapiro-Wilk检验: 统计量=0.9263, p值=0.1668  
Kolmogorov-Smirnov检验: 统计量=0.1627, p值=0.6689  
结论: 数学成绩成绩符合正态分布  
  
语文成绩统计:  
平均值: 78.67  
标准差: 10.80  
偏度: -0.5831  
峰度: -0.3500  
  
数学成绩统计:  
平均值: 77.33  
标准差: 13.50  
偏度: -0.7411  
峰度: -0.3650
```

设计说明:

1. 统计指标计算

皮尔逊相关系数: 衡量线性相关程度

斯皮尔曼相关系数: 衡量单调相关程度

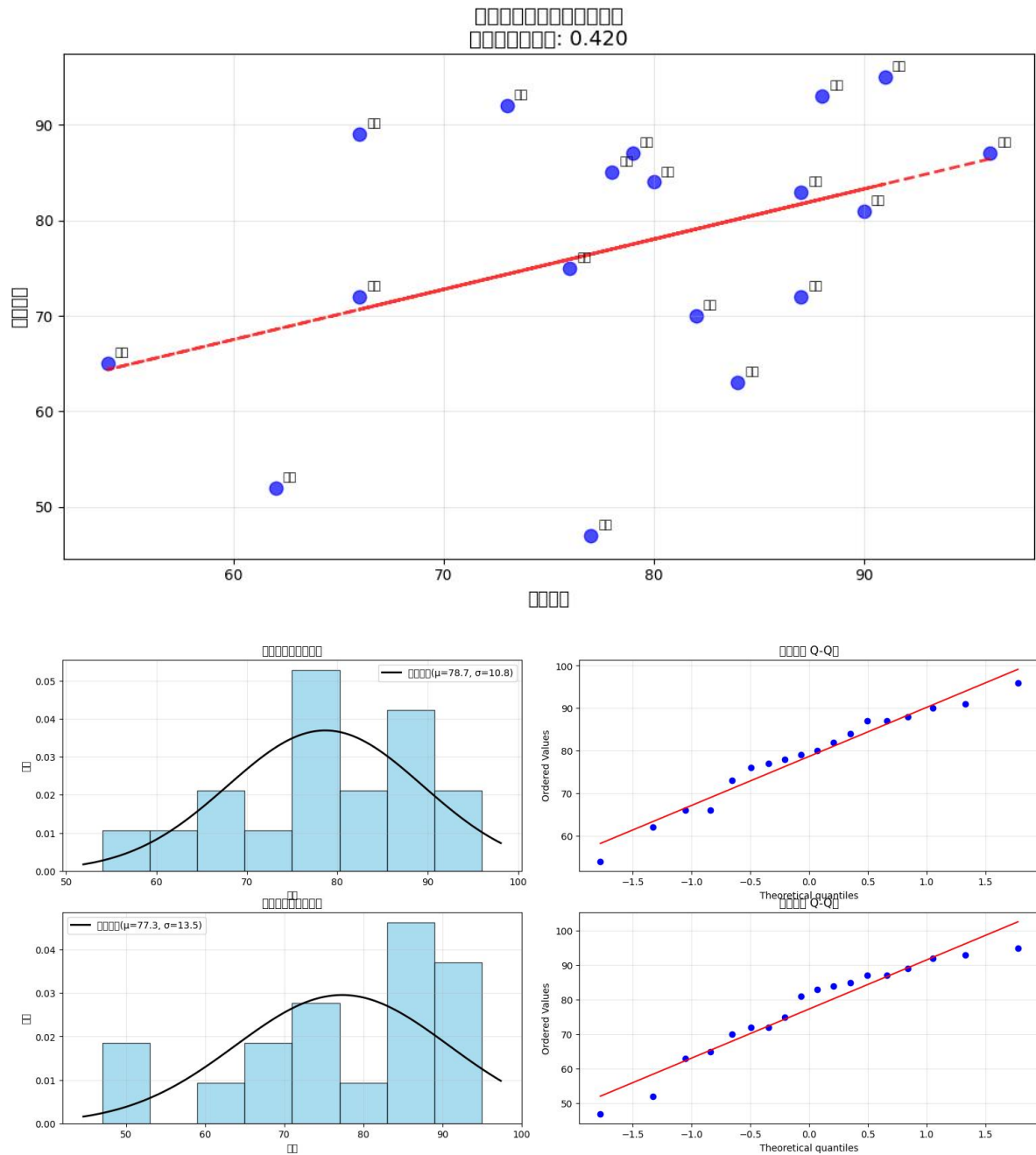
协方差: 衡量两变量变化趋势

2. 数据可视化部分

散点图: 展示两门成绩的分布关系

回归线: 显示线性趋势

学生姓名标注: 便于识别具体学生



3. 结果分析:

语文成绩正态性检验结果:

Shapiro-Wilk检验: 统计量=0.9599, p值=0.5999

Kolmogorov-Smirnov检验: 统计量=0.1247, p值=0.9102

结论: 语文成绩成绩符合正态分布

数学成绩正态性检验结果:

Shapiro-Wilk检验: 统计量=0.9263, p值=0.1668

Kolmogorov-Smirnov检验: 统计量=0.1627, p值=0.6689

结论: 数学成绩成绩符合正态分布

语文成绩统计:

平均值: 78.67

标准差: 10.80

偏度: -0.5831

峰度: -0.3500

数学成绩统计:

平均值: 77.33

标准差: 13.50

偏度: -0.7411

峰度: -0.3650

4. 图表分析:

(1) 散点图设计

数据点: 蓝色散点, 透明度0.7, 大小80像素

回归线: 红色虚线, 透明度0.8, 线宽2像素

网格: 浅灰色网格, 透明度0.3

标签: 每个学生姓名标注, 便于识别

可视化效果: 散点图显示语文成绩(X轴)与数学成绩(Y轴)的关系, 回归线显示两科成绩的线性趋势, 图表标题包含皮尔逊相关系数值, 学生姓名标注帮助识别异常值或特殊模式

(2) 分布分析可视化

第一行: 语文成绩分析

第二行: 数学成绩分析

每行包含直方图和Q-Q图

直方图设计:

分箱数: 8个分箱, 适合18个数据点

密度曲线: 叠加正态分布曲线(黑色实线)

图例: 显示正态分布的 μ 和 σ 参数

Q-Q图设计:

理论分位数对比实际分位数

如果数据符合正态分布, 点应大致落在对角线上

网格辅助判断偏离程度